Outline

5.1 Introduction
5.2 Modularizing Programs in C
5.3 Math Library Functions
5.4 Functions
5.5 Function Definitions
5.5.1 `square` Function
5.5.2 `maximum` Function
5.6 Function Prototypes: A Deeper Look
5.7 Function-Call Stack and Stack Frames
5.8 Headers
5.9 Passing Arguments by Value and by Reference
5.10 Random-Number Generation
5.11 Game Simulation Case Study: Rock, Paper, Scissors
5.12 Storage Classes
5.13 Scope Rules
5.14 Recursion
5.15 Example Using Recursion: Fibonacci Series
5.16 Recursion vs. Iteration
5.17 Secure C Programming—Secure Random-Number Generation

Summary | *Self-Review Exercises* | *Answers to Self-Review Exercises* | *Exercises*

5.1 Introduction

Most computer programs that solve real-world problems are much larger than those presented in the first few chapters. Experience has shown that the best way to develop and maintain a program is to construct it from smaller pieces, each of which is more manageable than the original program. This technique is called **divide and conquer**. We'll describe some key C features for designing, implementing, operating and maintaining large programs.

5.2 Modularizing Programs in C

In C, you use **functions** to modularize programs by combining the new functions you write with prepackaged **C standard library** functions. The C standard library provides a rich collection of functions for performing common mathematical calculations, string manipulations, character manipulations, input/output and many other useful operations. Prepackaged functions make your job easier because they provide many of the capabilities you need.

The C standard includes the C language and its standard library—standard C compilers implement both.[1] The functions `printf`, `scanf` and `pow` that we've used in previous chapters are from the standard library.

SE

Avoid Reinventing the Wheel

Familiarize yourself with the rich collection of C standard library functions to help reduce program-development time. When possible, use standard functions instead of writing new ones. The C standard library functions are written by experts, well tested

1. Some C standard library portions are designated as optional and are not available in all standard C compilers.

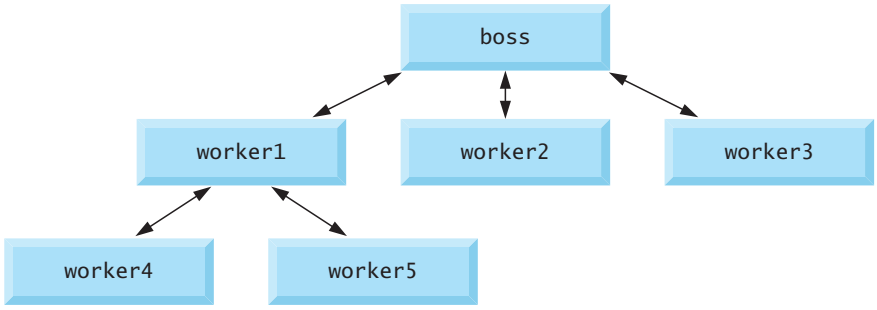
and efficient. Also, using the functions in the C standard library helps make programs more portable. PERF

Defining Functions

You can define functions to perform specific tasks that may be used at many points in a program. The statements defining the function are written once and are hidden from other functions. As we'll see, such hiding is crucial to good software engineering.

Calling and Returning from Functions

Functions are **invoked** by a **function call**, which specifies the function name and provides information (as arguments) that the function needs to perform its designated task.[2] A common analogy for this is the hierarchical form of management. A boss (the **calling function** or **caller**) asks a worker (the **called function**) to perform a task and report back when it's done. For example, a function that displays data on the screen calls the worker function `printf` to perform that task. Function `printf` displays the data and reports back—or **returns**—to the caller when it completes its task. The boss function does not know how the worker function performs its designated task. The worker may call other worker functions, and the boss will be unaware of this. The following diagram shows a boss function hierarchically communicating with several worker functions:



Note that `worker1` acts as a boss function to `worker4` and `worker5`. Relationships among functions may differ from the hierarchical structure shown in this figure.

Self Check

1 *(Fill-In)* Programs are typically written by combining new functions you write with prepackaged functions available in the _____.
Answer: C standard library.

2 *(Fill-In)* Functions are invoked by a function _____, which specifies the function name and provides information (as arguments) that the function needs to perform its designated task.
Answer: call.

2. In Chapter 7, Pointers, we'll discuss function pointers. You'll see that you also can call a function through a function pointer, and that you can actually pass functions to other functions.

5.3 Math Library Functions

C's math library functions (header `math.h`) allow you to perform common mathematical calculations. We use many of these functions in this section. To calculate and print the square root of `900.0` you might write

```
printf("%.2f", sqrt(900.0));
```

When this statement executes, it calls the math library function `sqrt` to calculate the square root of `900.0`, then prints the result as `30.00`. The `sqrt` function takes an argument of type `double` and returns a result of type `double`. In fact, all functions in the math library that return floating-point values return the data type `double`. Note that `double` values, like `float` values, can be output using the `%f` conversion specification. You may store a function call's result in a variable for later use as in

```
double result = sqrt(900.0);
```

Function arguments may be constants, variables, or expressions. If `c = 13.0`, `d = 3.0` and `f = 4.0`, then the statement

```
printf("%.2f", sqrt(c + d * f));
```

calculates the square root of `13.0 + 3.0 * 4.0 = 25.0` and prints it as `5.00`.

The following table summarizes several C math library functions. In the table, the variables `x` and `y` are of type `double`. The C11 standard added complex-number capabilities via the `complex.h` header.

Function	Description	Example
`sqrt(x)`	square root of x	`sqrt(900.0)` is `30.0` `sqrt(9.0)` is `3.0`
`cbrt(x)`	cube root of x (C99 and C11 only)	`cbrt(27.0)` is `3.0`[5] `cbrt(-8.0)` is `-2.0`
`exp(x)`	exponential function e^x	`exp(1.0)` is `2.718282` `exp(2.0)` is `7.389056`
`log(x)`	natural logarithm of x (base e)	`log(2.718282)` is `1.0` `log(7.389056)` is `2.0`

`log10(x)`	logarithm of x (base 10)	`log10(1.0)` is `0.0`
		`log10(10.0)` is `1.0`
		`log10(100.0)` is `2.0`

`fabs(x)`	absolute value of x as a floating-point number	`fabs(13.5)` is `13.5`
		`fabs(0.0)` is `0.0`
		`fabs(-13.5)` is `13.5`

`ceil(x)`	rounds x to the smallest integer not less than x	`ceil(9.2)` is `10.0` `ceil(-9.8)` is `-9.0`
`floor(x)`	rounds x to the largest integer not greater than x	`floor(9.2)` is `9.0` `floor(-9.8)` is `-10.0`
`pow(x, y)`	x raised to power y (x^y)	`pow(2, 7)` is `128.0` `pow(9, .5)` is `3.0`

Function	Description	Example
`fmod(x, y)`	remainder of *x/y* as a floating-point number	`fmod(13.657, 2.333)` is `1.992`
`sin(x)`	trigonometric sine of *x* (*x* in radians)	`sin(0.0)` is `0.0`
`cos(x)`	trigonometric cosine of *x* (*x* in radians)	`cos(0.0)` is `1.0`
`tan(x)`	trigonometric tangent of *x* (*x* in radians)	`tan(0.0)` is `0.0`

Self Check

1 *(Multiple Choice)* Which of the following statements a), b) or c) is *false*?

a) You call a function by writing its name followed by its argument, or a comma-separated list of arguments, in parentheses.

b) The following statement calculates and stores the square root of `900.0`:

```
double result = sqrt(900.0);
```

c) To use the math library functions, you must include the `math.h` header.

d) All of the above statements are *true*.

Answer: d.

2 *(True/False)* Function arguments may be constants, variables or expressions. If `c = 16.0`, `d = 4.0` and `f = 5.0`, then the following statement calculates and prints the square root of `100.00`:

```
printf("%.2f", sqrt(c + d * f));
```

Answer: *False*. Actually, it calculates the square root of `36.0` and prints it as `6.00`.

5.4 Functions

Functions allow you to modularize a program. In programs containing many functions, `main` is often implemented as a group of calls to functions that perform the bulk of the program's work. SE

Functionalizing Programs

There are several motivations for "functionalizing" a program. The **divide-and-conquer** approach makes program development more manageable. Another motivation is building new programs by using existing functions. Such **software reusability** is a key concept in object-oriented programming languages derived from C, such as C++, Java, C# (pronounced "C sharp"), Objective-C and Swift.

With good function naming and definition, you can create programs from standardized functions that accomplish specific tasks, rather than custom code. This is known as **abstraction**. We use abstraction each time we use standard library functions like `printf`, `scanf` and `pow`. A third motivation is to avoid repeating code in a program. Packaging code as a function allows it to be executed from other program locations by calling that function.

Each function should be limited to performing a single, well-defined task, and the function name should express that task. This facilitates abstraction and promotes SE

SE software reuse. If you cannot choose a concise name to describe what the function does, it may be performing too many diverse tasks. It's usually best to break such a function into smaller functions—a process called *decomposition*.

Self Check

1 *(Fill-In)* There are several motivations for "functionalizing" a program. The divide-and-conquer approach makes program development more manageable. Another motivation is _____—using existing functions as building blocks to create new programs.
Answer: software reusability.

2 *(True/False)* Each function should perform a rich collection of related tasks, and the function name should describe those tasks.
Answer: *False*. Actually, each function should be limited to performing a single, well-defined task, and the function name should describe that task. This facilitates abstraction and promotes software reuse.

5.5 Function Definitions

Each program we've presented has consisted of a function called `main` that called standard library functions to accomplish its tasks. Now we consider how to write custom functions.

5.5.1 square Function

Consider a program that uses a function `square` to calculate and print the squares of the integers from 1 to 10 (Fig. 5.1).

```
// fig05_01.c
// Creating and using a function.
#include <stdio.h>

int square(int number); // function prototype

int main(void) {
   // loop 10 times and calculate and output square of x each time
   for (int x = 1; x <= 10; ++x) {
      printf("%d  ", square(x)); // function call
   }

   puts("");
}

// square function definition returns the square of its parameter
int square(int number) { // number is a copy of the function's argument
   return number * number; // returns square of number as an int
}
```

Fig. 5.1 | Creating and using a function. (Part 1 of 2.)

```
1  4  9  16  25  36  49  64  81  100
```

Fig. 5.1 | Creating and using a function. (Part 2 of 2.)

Calling Function square

Function `square` is **invoked** or **called** in `main` within the `printf` statement (line 10):

```
printf("%d  ", square(x)); // function call
```

Function `square` receives a copy of the argument `x`'s value in the parameter `number` (line 17). Then `square` calculates `number * number` and passes the result back to line 10 in `main` where `square` was invoked. Line 10 passes `square`'s result to function `printf`, which displays the result on the screen. This process repeats 10 times—once for each iteration of the `for` statement.

square Function Definition

Function `square`'s definition (lines 17–19) shows that it expects an `int` parameter `number`. The keyword `int` preceding the function name (line 17) indicates that `square` returns an integer result. The **`return` statement** in `square` passes the result of `number * number` back to the calling function.

Choosing meaningful function names and meaningful parameter names makes programs more readable and helps avoid excessive comments. Programs should be written as collections of small functions. This makes programs easier to write, debug, maintain, modify and reuse. SE

A function requiring a large number of parameters may be performing too many tasks. Consider dividing the function into smaller functions that perform the separate tasks. The function's return type, name and parameter list should fit on one line if possible. SE

Local Variables

All variables defined in function definitions are **local variables**—they can be accessed only in the function in which they're defined. Most functions have **parameters** that enable communicating between functions via arguments in function calls. A function's parameters are also local variables of that function.

square Function Prototype

Line 5

```
int square(int number); // function prototype
```

is a **function prototype**. The `int` in parentheses informs the compiler that `square` expects to receive an integer value from the caller. The `int` to the left of the function name `square` informs the compiler that `square` returns an integer result to the caller. Forgetting the semicolon at the end of a function prototype is a syntax error. ERR

The compiler compares `square`'s call (line 10) to its prototype to ensure that:

- the number of arguments is correct,

- the arguments are of the correct types,
- the argument types are in the correct order, and
- the return type is consistent with the context in which the function is called.

The function prototype, first line of the function definition and function calls should all agree in the number, type and order of arguments and parameters. The function prototype and function header must have the same return type, which affects where the function can be called. For example, a function with the return type `void` cannot be used in an assignment statement to store a value or in a call to `printf` to display a value. Function prototypes are discussed in detail in Section 5.6.

Format of a Function Definition

The format of a function definition is

```
return-value-type function-name(parameter-list) {
    statements
}
```

The *function-name* is any valid identifier. The ***return-value-type*** is the type of the result returned to the caller. The *return-value-type* `void` indicates that a function does *not* return a value. Together, the *return-value-type*, *function-name* and *parameter-list* are sometimes referred to as the function **header**.

The ***parameter-list*** is a comma-separated list that specifies the parameters received by the function when it's called. If a function does not receive any parameters, *parameter-list* should contain the keyword `void`. Each parameter must include its type; otherwise, a compilation error occurs.

ERR

Placing a semicolon after the *parameter-list*'s right parenthesis in a function definition is an error, as is redefining a parameter as a local variable in a function. Although it's not incorrect to do so, do not use the same names for a function's arguments and the corresponding parameters in the function definition—this helps avoid ambiguity.

ERR

Function Body

The *statements* within braces form the **function body**, which also is a block. Local variables can be declared in any block, and blocks can be nested. Functions cannot be nested—defining a function inside another function is a syntax error.

ERR

Returning Control from a Function

There are three ways to return control from a called function to the point at which a function was invoked. If the function does not return a result, control is returned simply when the function-ending right brace is reached, or by executing the statement

```
return;
```

If the function does return a result, the statement

```
return expression;
```

returns the *expression*'s value to the caller.

main's Return Type

The main function's int return value indicates whether the program executed correctly. In earlier versions of C, we'd explicitly place

```
return 0; // 0 indicates successful program termination
```

at the end of main. The C standard indicates that main implicitly returns 0 if you omit the preceding statement—as we do throughout this book. You can explicitly return nonzero values from main to indicate that a problem occurred during your program's execution. For information on how to report a program failure, see the documentation for your particular operating system.

5.5.2 maximum Function

Let's consider a custom maximum function that returns the largest of three integers (Fig. 5.2). Next, they're passed to maximum (line 17), which determines the largest integer. This value is returned to main by the return statement in maximum (line 32). The printf statement in line 17 then prints the value returned by maximum.

```
1  // fig05_02.c
2  // Finding the maximum of three integers.
3  #include <stdio.h>
4
5  int maximum(int x, int y, int z); // function prototype
6
7  int main(void) {
8     int number1 = 0; // first integer entered by the user
9     int number2 = 0; // second integer entered by the user
10    int number3 = 0; // third integer entered by the user
11
12    printf("%s", "Enter three integers: ");
13    scanf("%d%d%d", &number1, &number2, &number3);
14
15    // number1, number2 and number3 are arguments
16    // to the maximum function call
17    printf("Maximum is: %d\n", maximum(number1, number2, number3));
18 }
19
20 // Function maximum definition
21 int maximum(int x, int y, int z) {
22    int max = x; // assume x is largest
23
24    if (y > max) { // if y is larger than max,
25       max = y; // assign y to max
26    }
27
28    if (z > max) { // if z is larger than max,
29       max = z; // assign z to max
30    }
31
32    return max; // max is largest value
33 }
```

Fig. 5.2 | Finding the maximum of three integers. (Part 1 of 2.)

```
Enter three integers: 22 85 17
Maximum is: 85
```

```
Enter three integers: 47 32 14
Maximum is: 47
```

```
Enter three integers: 35 8 79
Maximum is: 79
```

Fig. 5.2 | Finding the maximum of three integers. (Part 2 of 2.)

The function initially assumes that its first argument (stored in the parameter `x`) is the largest and assigns it to `max` (line 22). Next, the `if` statement at lines 24–26 determines whether `y` is greater than `max` and, if so, assigns `y` to `max`. Then, the `if` statement at lines 28–30 determines whether `z` is greater than `max` and, if so, assigns `z` to `max`. Finally, line 32 returns `max` to the caller.

Self Check

1 *(Multiple Choice)* The following line of code is a _____.

```
int square(int y);
```

a) Function definition.
b) Function statement.
c) Function prototype.
d) None of the above.

Answer: c.

2 *(Multiple Choice)* Consider the maximum function in Fig. 5.2. Which of the following statements is *false*?

a) The code determines the largest of three integer values.
b) The statement `return max;` sends the result back to the calling function.
c) The code in line 21—`int maximum(int x, int y, int z)`—is commonly called a function header.
d) If `int max = x;` (line 22) were accidentally replaced by `int max = y;` the function would still return the same result.

Answer: d) is *false*. The function would then incorrectly return the larger of the values contained in only parameters `y` and `z`.

5.6 Function Prototypes: A Deeper Look

An important C feature is the function prototype, which was borrowed from C++. The compiler uses function prototypes to validate function calls. Pre-standard C did *not* perform this kind of checking, so it was possible to call functions improperly

without the compiler detecting the errors. Such calls could result in fatal execution-time errors or nonfatal errors that caused subtle, difficult-to-detect problems. Function prototypes correct this deficiency.

You should include function prototypes for all functions to take advantage of C's type-checking capabilities. Use `#include` preprocessor directives to obtain function prototypes from standard library headers, third-party library headers and headers for functions developed by you or your team members. SE

The function prototype for `maximum` in Fig. 5.2 (line 5) is

```
int maximum(int x, int y, int z); // function prototype
```

It states that `maximum` takes three arguments of type `int` and returns an `int` result. Notice that the function prototype (omitting the semicolon) is the same as the `maximum` definition's first line. We include parameter names in function prototypes for documentation purposes. The compiler ignores these names, so the following prototype also is valid: SE

```
int maximum(int, int, int);
```

Compilation Errors

A function call that does not match the function prototype is a compilation error. It also is an error if the function prototype and the function definition disagree. For example, in Fig. 5.2, if the function prototype had been written ERR

```
void maximum(int x, int y, int z);
```

the compiler would generate an error, because the function prototype's `void` return type would differ from the `int` return type in the function header.

Argument Coercion and "Usual Arithmetic Conversion Rules"

Another important feature of function prototypes is **argument coercion**, i.e., implicitly converting arguments to the appropriate type. For example, calling the math library function `sqrt` with an integer argument still works even though the function prototype in `<math.h>` specifies a `double` parameter. The following statement correctly evaluates `sqrt(4)` and prints `2.000`:

```
printf("%.3f\n", sqrt(4));
```

The function prototype causes the compiler to convert a copy of the `int` value `4` to the `double` value `4.0` before passing it to `sqrt`. In general, argument values that do not correspond precisely to the function prototype's parameter types are converted to the proper type before the function is called. Such conversions can lead to incorrect results if C's **usual arithmetic conversion rules** are not followed. These rules specify how values can be converted to other types without losing data. ERR

In our `sqrt` example, an `int` is automatically converted to a `double` without changing its value—`double` can represent a much wider range of values than `int`. However, a `double` converted to an `int` truncates the `double`'s fractional part, thus changing the original value. Converting large integer types to small integer types (e.g., `long` to `short`) can also change values.

Mixed-Type Expressions

The usual arithmetic conversion rules are handled by the compiler. They apply to **mixed-type expressions**—that is, expressions containing values of multiple data types. In such expressions, the compiler makes temporary copies of values that need to be converted, then converts the *copies* to the "highest" type in the expression—this is known as **promotion**. For mixed-type expressions containing at least one floating-point value:

- If one value is a `long double`, the other values are converted to `long double`.
- If one value is a `double`, the other values are converted to `double`.
- If one value is a `float`, the other values are converted to `float`.

If the mixed-type expression contains only integer types, then the usual arithmetic conversions specify a set of integer promotion rules.

Section 6.3.1 of the C standard document specifies the complete details of arithmetic operands and the usual arithmetic conversion rules. The following table lists the floating-point and integer data types with each type's `printf` and `scanf` conversion specifications. In most cases, the integer types lower in the following table are converted to higher types:

Data type	`printf` conversion specification	`scanf` conversion specification
Floating-point types		
`long double`	`%Lf`	`%Lf`
`double`	`%f`	`%lf`
`float`	`%f`	`%f`
Integer types		
`unsigned long long int`	`%llu`	`%llu`
`long long int`	`%lld`	`%lld`
`unsigned long int`	`%lu`	`%lu`
`long int`	`%ld`	`%ld`
`unsigned int`	`%u`	`%u`
`int`	`%d`	`%d`
`unsigned short`	`%hu`	`%hu`
`short`	`%hd`	`%hd`
`char`	`%c`	`%c`

A value can be converted to a lower type only by explicitly assigning the value to a variable of lower type or by using a cast operator. Arguments are converted to the parameter types specified in a function prototype as if the arguments were being assigned to variables of those types. So, if we pass a `double` to our `square` function in Fig. 5.1, the `double` is converted to `int` (a lower type), and `square` usually returns an incorrect value. For example, `square(4.5)` returns `16`, not `20.25`.

ERR ⊗ Converting from a higher data type in the promotion hierarchy to a lower type can change the data value. Many compilers issue warnings in such cases.

Function Prototype Notes

If there's no function prototype for a function, the compiler forms one from the first occurrence of the function—either the function definition or a call to the function. This typically leads to warnings or errors, depending on the compiler. ERR

Always include function prototypes for the functions you define or use in your program to help prevent compilation errors and warnings. SE

A function prototype placed outside any function definition applies to all calls to the function appearing after the function prototype. A function prototype placed in a function body applies only to calls in that function made after that prototype. SE

Self Check

1 *(Fill-In)* In a mixed-type expression, the compiler makes a temporary copy of each value that needs to be converted, then converts the copies to the "highest" type in the expression—this is known as _____.

Answer: promotion.

2 *(Multiple Choice)* Consider the following function prototype for a `maximum` function:

```
int maximum(int x, int y, int z); // function prototype
```

Which of the following statements is false?

a) It states that `maximum` takes three arguments of type `int` and returns a result of type `int`.
b) Parameter names are required in function prototypes.
c) A function's prototype is often the same as the function's header, except that the header does not end in a semicolon.
d) Forgetting the semicolon at the end of a function prototype is a syntax error.

Answer: b) is *false*. Parameter names in function prototypes are for documentation purposes. The compiler ignores these names, so `int maximum(int, int, int);` is equivalent to the preceding prototype.

5.7 Function-Call Stack and Stack Frames

To understand how C performs function calls, we first need to consider a data structure (i.e., collection of related data items) known as a **stack**. Think of a stack as analogous to a pile of dishes. You usually place a dish at the top—referred to as **pushing** the dish onto the stack. Similarly, you typically remove a dish from the top—referred to as **popping** the dish off the stack. Stacks are known as **last-in, first-out (LIFO) data structures**—the last item pushed (inserted) on the stack is the *first* item popped (removed) from the stack.

Function-Call Stack

An important mechanism for computing students to understand is the **function-call stack** (sometimes referred to as the **program execution stack**). This data structure—

working "behind the scenes"—supports the function call/return mechanism. As you'll see in this section, the function-call stack also supports creating, maintaining and destroying each called function's local variables.

Stack Frames

As each function is called, it may call other functions, which may call other functions—all before any function returns. Each function eventually must return control to its caller. So, we must keep track of the return addresses that each function needs to return control to the function that called it. The function-call stack is the perfect data structure for handling this information. Each time a function calls another function, an entry is pushed onto the stack. This entry, called a **stack frame**, contains the *return address* that the called function needs in order to return to the calling function. It also contains some additional information we'll soon discuss. When a called function returns, the stack frame for the function call is popped, and control transfers to the return address specified in the popped stack frame.

Each called function always finds at the *top* of the call stack the information it needs to return to its caller. If a called function calls another function, a stack frame for the new function call is pushed onto the call stack. Thus, the return address required by the newly called function to return to its caller is now located at the top of the stack.

The stack frames have another important responsibility. Most functions have local variables, which must exist while a function is executing. They need to remain active if the function makes calls to other functions. But when a called function returns to its caller, the called function's local variables need to "go away." The called function's stack frame is a perfect place to reserve the memory for local variables. That stack frame exists only as long as the called function is active. When that function returns—and no longer needs its local variables—its stack frame is popped from the stack. Those local variables are no longer known to the program.

Stack Overflow

Of course, the amount of memory in a computer is finite, so only limited memory can be used to store stack frames on the function-call stack. If more function calls occur than can have their stack frames stored on the function-call stack, a *fatal* error known as **stack overflow**[3] occurs.

Function-Call Stack in Action

Now let's consider how the call stack supports the operation of a `square` function called by `main` (lines 8–12 of Fig. 5.3).

3. This is how the website `stackoverflow.com` got its name—a popular website for getting answers to your programming questions.

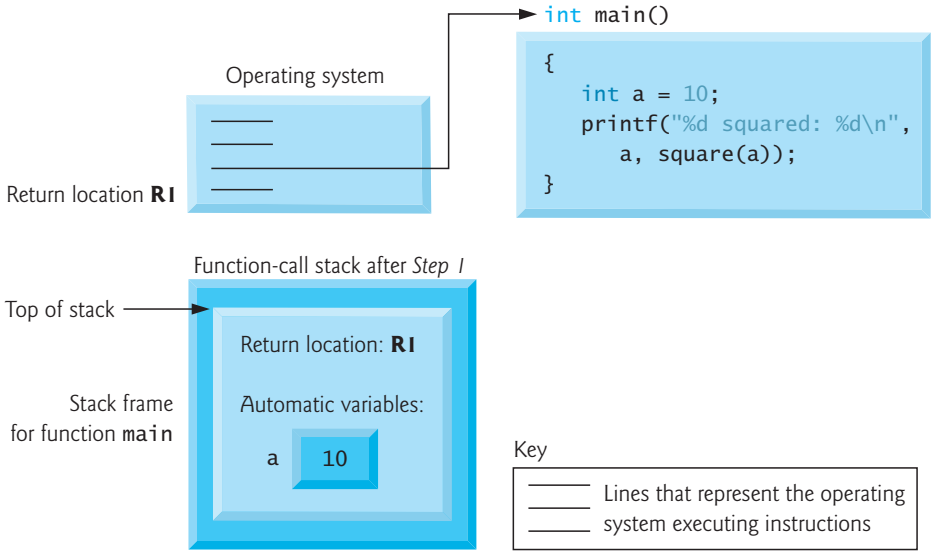
```c
1  // fig05_03.c
2  // Demonstrating the function-call stack
3  // and stack frames using a function square.
4  #include <stdio.h>
5
6  int square(int x); // prototype for function square
7
8  int main() {
9     int a = 10; // value to square (local variable in main)
10
11    printf("%d squared: %d\n", a, square(a)); // display a squared
12 }
13
14 // returns the square of an integer
15 int square(int x) { // x is a local variable
16    return x * x; // calculate square and return result
17 }
```

```
10 squared: 100
```

Fig. 5.3 | Demonstrating the function-call stack and stack frames using a function `square`.

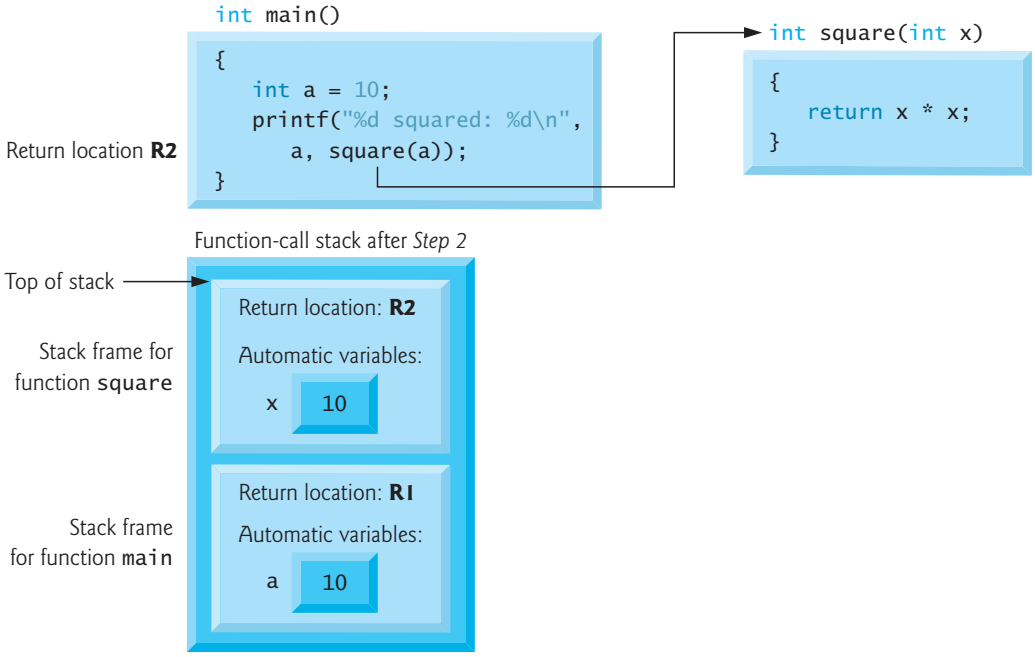
Step 1: Operating System Invokes `main` to Execute Application

First, the operating system calls `main`—this pushes a stack frame onto the stack (as shown in the following diagram). The stack frame tells `main` how to return to the operating system (that is, transfer to return address `R1`) and contains the space for `main`'s local variable a, which is initialized to `10`.



Step 2: `main` Invokes Function `square` to Perform Calculation

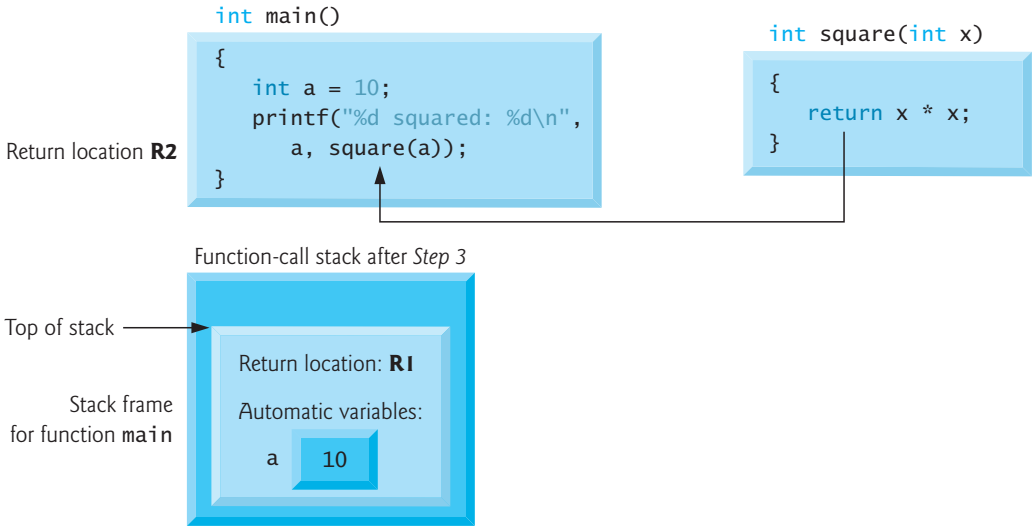
Function `main`—before returning to the operating system—now calls function `square` in line 11 of Fig. 5.3. This causes a stack frame for `square` (lines 15–17) to be pushed onto the function-call stack, as shown in the following diagram:



This stack frame contains the return address that `square` needs to return to `main` (i.e., `R2`) and the memory for `square`'s local variable (i.e., `x`).

Step 3: `square` Returns Its Result to `main`

After `square` calculates the square of its argument, it needs to return to `main`—and no longer needs the memory for its local variable `x`. So the stack is popped—giving `square` the return location in `main` (i.e., `R2`) and losing `square`'s local variable. The following diagram shows the function-call stack *after* `square`'s stack frame has been popped:



Function `main` now displays the result of calling `square` (line 11 in Fig. 5.3). Reaching `main`'s closing right brace pops its stack frame from the stack. This gives `main` the address it needs to return to the operating system (i.e., `R1` in the preceding diagram). At this point, the memory for `main`'s local variable (i.e., `a`) is unavailable.

Flaw in Our Discussion

There is a flaw in the preceding discussion and diagrams. We showed `main` calling out to `square` and `square` returning to `main`, but, of course, `printf` is a function too. As you study the code in Fig. 5.3, you might be inclined to say that `main` calls `printf`, then `printf` calls `square`. However, `printf`'s argument values must be known in full before `printf` can be called. So execution proceeds as follows:

1. The operating system calls `main`, so `main`'s stack frame is pushed onto the stack.
2. `main` calls `square`, so `square`'s stack frame is pushed onto the stack.
3. `square` calculates and returns to `main` a value for use in `printf`'s argument list, so `square`'s stack frame is popped from the stack.
4. `main` calls `printf`, so `printf`'s stack frame is pushed onto the stack.
5. `printf` displays its arguments, then returns to `main`, so `printf`'s stack frame is popped from the stack.
6. `main` terminates, so `main`'s stack frame is popped from the stack.

Data Structures

You've now seen how valuable the stack data structure is in implementing a key mechanism that supports program execution. Data structures have many important applications in computer science. We discuss stacks, queues, lists and trees in Chapter 12.

Self Check

1 *(Fill-In)* Each time a function calls another function, an entry is pushed onto the stack. This entry, called a _____, contains the return address that the called function needs in order to return to the calling function.
Answer: stack frame.

2 *(True/False)* A called function's stack frame is a perfect place to reserve the memory for local variables. That stack frame exists only as long as the called function is active. When the function returns—and no longer needs its local variables—its stack frame is popped from the stack. At that point, those local variables are no longer known to the program.
Answer: *True*.

5.8 Headers

Each standard library has a corresponding **header** containing the function prototypes for all the functions in that library and definitions of various data types and constants needed by those functions. The following table alphabetically lists several standard library headers that may be included in programs. The C standard includes additional headers. The term "macros"—used several times in this table—is discussed in detail in Chapter 14.

Header	Explanation
Headers we use or discuss in this book:	
`<assert.h>`	Contains information for adding diagnostics that aid program debugging.
`<ctype.h>`	Contains function prototypes for functions that test characters for certain properties, and function prototypes for functions that can be used to convert lowercase letters to uppercase letters and vice versa.
`<float.h>`	Contains the floating-point size limits of the system.
`<limits.h>`	Contains the integral size limits of the system.
`<math.h>`	Contains function prototypes for math library functions.
`<signal.h>`	Contains function prototypes and macros to handle various conditions that may arise during program execution.
`<stdarg.h>`	Defines macros for dealing with a list of arguments to a function whose number and types are unknown.
`<stdio.h>`	Contains function prototypes for the standard input/output library functions and information used by them.
`<stdlib.h>`	Contains function prototypes for conversions of numbers to text and text to numbers, memory allocation, random numbers and other utility functions.
`<string.h>`	Contains function prototypes for string-processing functions.
`<time.h>`	Contains function prototypes and types for manipulating the time and date.
Other headers:	
`<errno.h>`	Defines macros that are useful for reporting error conditions.
`<locale.h>`	Contains function prototypes and other information that enables a program to be modified for the current locale on which it's running. The locale notion enables the computer system to handle different conventions for expressing data such as dates, times, currency amounts and large numbers throughout the world.
`<setjmp.h>`	Contains function prototypes for functions that allow bypassing of the usual function call and return sequence.
`<stddef.h>`	Contains common type definitions used by C.

You can create custom headers. A programmer-defined header can be included by using the `#include` preprocessor directive. For example, if the prototype for our square function was located in the header `square.h`, we'd include that header in our program by using the following directive at the top of the program:

```
#include "square.h"
```

Section 14.2 presents additional information on including headers, such as why programmer-defined headers are enclosed in quotes (`""`) rather than angle brackets (`<>`).

Self Check

1 *(Fill-In)* The _____ header contains function prototypes for string-processing functions.

Answer: `<string.h>`.